

Week 7 | 实验 | 把 Week3 文档资产升级成 Week8-ready 文档资产

本页结构

跑出 parse → section → chunk → anchor → quality → gate 的最小闭环	1
这次实验要完成什么	1
参考实验时间	2
本次实验产出	2
怎么看输出，不只看命令成功	2
先看一张闭环图	3
实验前置检查	3
0. 环境准备	3
1. 确认 Week3 文档输入	3
2. schema / contract tests	4
3. Docling-first parse dry-run	4
4. 正常闭环运行	4
5. 检查 sections	5
6. 检查 chunks	5
7. 检查 evidence anchors	6
8. 检查 quality report 与 Week8 ready gate	6
9. bad case review	6
10. 常见错误与排查	7
提交清单	7
实验自检表	7
延伸阅读	7
Footnotes	8
实验最重要判断	8

跑出 parse → section → chunk → anchor → quality → gate 的最小闭环

这次实验不是做一个独立 PDF demo。

你要从 Week3 的 raw document asset 出发，把它升级成能被 Week8 安全索引和引用的文档资产 v1。

这次实验要完成什么

- 确认 Week3 文档输入存在
- 从 MinIO / source fallback 读取 raw document
- 执行 Docling-first parse
- 生成 normalized sections
- 执行 section-aware chunking
- 生成 evidence anchors
- 输出 `chunk_quality_report.md`
- 输出 `week8_ready_gate.json`

参考实验时间

按课程统一节奏完成即可。重点是先跑通闭环，再整理 quality report、bad case review 和 Week8 handoff。

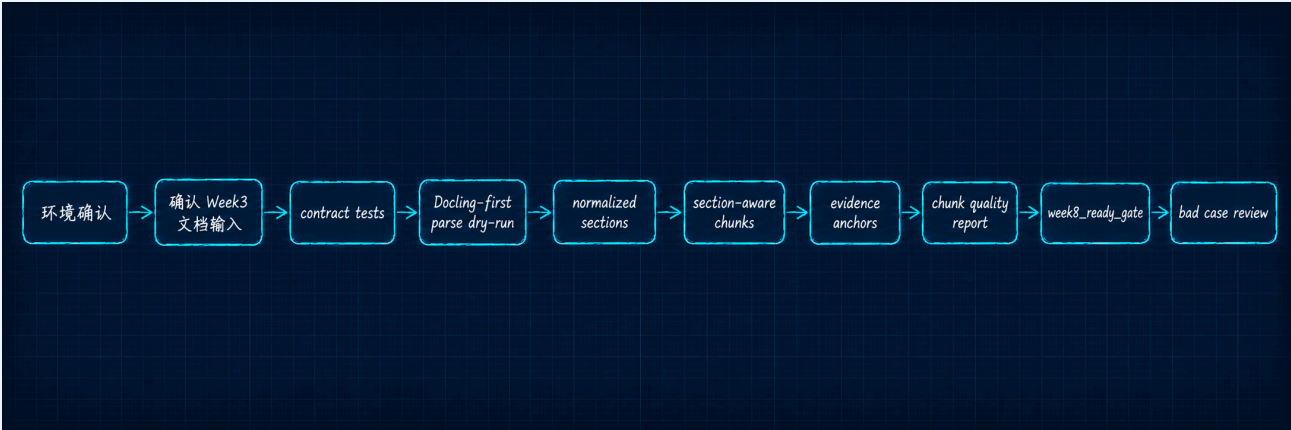
本次实验产出

- reports/week07/parse_run_report.json
- artifacts/week07/sections.json
- artifacts/week07/chunks.json
- artifacts/week07/evidence_anchors.json
- reports/week07/chunk_quality_samples.json
- reports/week07/chunk_quality_report.md
- reports/week07/week8_ready_gate.json
- reports/week07/bad_case_review.md
- reports/week07/week07_lab_execution_summary.md

怎么看输出，不只看命令成功

输出	小白看什么	工程验收点
parse_run_report.json	parser 有没有跑起来	parser backend、fallback、warning、strategy version 都写清
sections.json	文档结构有没有留下来	section_path、page_no、bbox / missing reason、source_fingerprint 完整
chunks.json	chunk 是否能被索引	稳定 chunk_id、非空 content、继承 metadata、quality flags
evidence_anchors.json	能不能回到原文	每个 chunk 至少一个 anchor, 能回到 source/page/section
chunk_quality_report.md	是否允许进入 Week8	hard / warn / N/A / quarantine 结论可解释
week8_ready_gate.json	Week8 怎么消费	allowed、blocked、warnings、consumption rule 明确

先看一张闭环图



实验前置检查

检查项	目的	如果失败
Docker Compose 能启动	保证 Student Core 不依赖宿主主机 venv	先修环境，不要跳到 parser
manifest 存在	保证 Week3 输入声明可读	回到 Week3 manifest / seed 数据检查
contract tests 通过	保证对象边界成立	先修 schema 和 fixtures
raw object 或 fallback 可解释	保证不是伪造输入	在 report 中写清 fallback reason

0. 环境准备

本实验默认 Docker-first 路径：

```
cp infra/env/.env.example infra/env/.env.local

docker compose --env-file infra/env/.env.local -f infra/docker-compose.yml up -d --build

docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    python --version
```

后续命令都走 `docker compose --profile tools ... devbox`，不要默认要求在宿主机创建 venv。

期望输出：

- devbox 能打印 Python 版本
- compose 服务处于 running / healthy
- 没有要求你在宿主机创建新的虚拟环境

1. 确认 Week3 文档输入

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    ls data/seed_manifests/manifest_workspace_helpcenter_v1.json data/canonization/documents
```

你要确认输入侧至少能说明：

- `manifest_path`
- `raw_object_path` 或本地 fallback source
- `source_fingerprint`
- `doc_id` 或 `source_id`
- 当前是否是 dry-run / synthetic fallback

验收点：如果当前只能走 fallback，必须在后续 `parse_run_report.json` 写出 `fallback_reason`，不能把 fallback 写成完整 raw object 成功。

2. schema / contract tests

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
  pytest tests/contract/test_week07_parse_contracts.py -v
```

通过以后再跑 parse。否则 sections、chunks、anchors 的对象边界还没立住。

验收点：contract test 不只是“测试绿了”，它证明 Week7 的 section / chunk / anchor 字段边界已经可被机器检查。

3. Docling-first parse dry-run

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.parse_normalize.run_parse \
    --manifest-path data/seed_manifests/manifest_workspace_helpcenter_v1.json \
    --parser auto \
    --chunk-strategy section_aware_v1 \
    --data-release-id week07-lab-local \
    --dry-run \
    --artifacts-dir artifacts/week07 \
    --report-json reports/week07/parse_run_report.json \
    --quality-report-md reports/week07/chunk_quality_report.md \
    --week8-gate-json reports/week07/week8_ready_gate.json
```

Dry-run 允许主项目在本地 raw source 不完整时给出 deterministic fallback，但必须写明 warning，不能伪装成完整 Docling 成功。

期望输出：

- `reports/week07/parse_run_report.json` 被生成
- report 中有 parser backend、strategy version、warnings
- dry-run 不应写入不可回滚的生产状态

4. 正常闭环运行

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.parse_normalize.run_parse \
    --manifest-path data/seed_manifests/manifest_workspace_helpcenter_v1.json \
    --parser auto \
    --chunk-strategy section_aware_v1 \
```

```
--data-release-id week07-lab-local \  
--artifacts-dir artifacts/week07 \  
--report-json reports/week07/parse_run_report.json \  
--quality-report-md reports/week07/chunk_quality_report.md \  
--week8-gate-json reports/week07/week8_ready_gate.json
```

如果 Docling 依赖未安装, CLI 应该给出明确安装建议, 而不是静默生成假数据。

期望输出:

- artifacts/week07/sections.json
- artifacts/week07/chunks.json
- artifacts/week07/evidence_anchors.json
- reports/week07/chunk_quality_report.md
- reports/week07/week8_ready_gate.json

5. 检查 sections

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run  
--rm devbox \  
jq '.sections[0]' artifacts/week07/sections.json
```

必须能看到类似字段:

- section_id
- doc_id
- section_path
- page_no
- bbox 或 bbox_missing_reason
- source_fingerprint
- doc_version
- parse_strategy_version

6. 检查 chunks

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run  
--rm devbox \  
jq '.chunks[0]' artifacts/week07/chunks.json
```

必须能看到:

- chunk_id
- section_id
- chunk_index
- content
- chunk_strategy_version
- overlap_prev
- overlap_next
- quality_flags

7. 检查 evidence anchors

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
jq '.anchors[0]' artifacts/week07/evidence_anchors.json
```

每个 chunk 至少应该有一个 anchor。Anchor 要能回到：

- source uri / raw object uri
- doc version
- source fingerprint
- page number
- bbox 或 missing reason
- section path

8. 检查 quality report 与 Week8 ready gate

报告至少包含：

- metadata completeness
- page reference rate
- bbox coverage
- anchor coverage
- empty chunk rate
- orphan chunk rate
- overlap duplicate rate
- PII leakage risk
- sample_shortfall

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
sed -n '1,120p' reports/week07/chunk_quality_report.md
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
jq '.' reports/week07/week8_ready_gate.json
```

9. bad case review

至少选一个失败或 warning 样本，写入：

```
reports/week07/bad_case_review.md
```

建议结构：

字段	要回答
bad case	哪个文档 / section / chunk 出问题
root cause	parser、chunking、anchor 还是 source 质量问题
impact	Week8 如果消费它会发生什么
fix	修 raw source、换 parser、调 chunking、隔离还是补 reason

字段	要回答
status	fixed / quarantined / accepted warning

10. 常见错误与排查

问题	可能原因	处理
找不到 <code>s3://</code> raw object	MinIO 未启动或 bucket/key 不存在	先查 manifest、MinIO bucket 和 <code>parse_run_report.json</code> warning
指纹不一致	读到的 raw bytes 不是 ingest 时那份文件	hard fail 或 quarantine, 不要继续
Docling import 失败	devbox 未安装 parse extra	安装 optional extra 或使用 fallback
PDF 没 page_no	parser 输出缺失关键 metadata	hard fail, 不能进入 Week8
HTML 没 bbox	正常 not applicable	报告里写 <code>non_paged_bbox_not_applicable_cou</code>

提交清单

- `reports/week07/parse_run_report.json`
- `artifacts/week07/sections.json`
- `artifacts/week07/chunks.json`
- `artifacts/week07/evidence_anchors.json`
- `reports/week07/chunk_quality_samples.json`
- `reports/week07/chunk_quality_report.md`
- `reports/week07/week8_ready_gate.json`
- `reports/week07/bad_case_review.md`
- `reports/week07/week07_lab_execution_summary.md`

实验自检表

问题	合格回答
这次 parse 用了哪份 raw document?	能指出 manifest、source URI / fallback 和 source_fingerprint
为什么 chunk 可以进入 Week8?	有 anchor、metadata 完整、quality gate 允许
哪些 chunk 不该索引?	blocked / quarantine / hard fail / 缺 anchor 的 chunk
如果 bbox 缺失怎么办?	按 content type 写 <code>bbox_missing_reason</code> , PDF 不能静默通过
如果样本不足 50 份怎么办?	写 <code>sample_shortfall</code> 和扩展抽样计划

延伸阅读

- Docling docs: <https://docling-project.github.io/docling/>。实验前先确认 parser 能力边界。

- JSON Schema: <https://json-schema.org/understanding-json-schema/>。理解 contract tests 为什么先于 runtime。
- Quarto Mermaid: <https://quarto.org/docs/authoring/diagrams.html#mermaid>。用于把实验链路图写清楚，不额外引入前端组件。

Footnotes

1. 本实验允许 deterministic fallback，但必须写清 fallback reason，不允许伪造 Week3 raw object 已完整可用。
2. `dry-run` 不是低价值模式；它是进入真实运行前检查输入、契约和输出形状的安全门。
3. Week7 实验的终点是 Week8-ready gate，不是 embedding 或 RAG API。

i 边界提醒

本实验不生成 embedding，不写 pgvector，不跑 hybrid retrieval，不启动 RAG API。你看到的所有输出都应该停在 Week8-ready document asset，而不是检索服务。

实验最重要判断

这次实验不是为了生成最多 chunks，而是为了证明：

Week8 只能消费带 evidence anchor 且通过质量门禁的 chunks。